# lowThrust — how it works

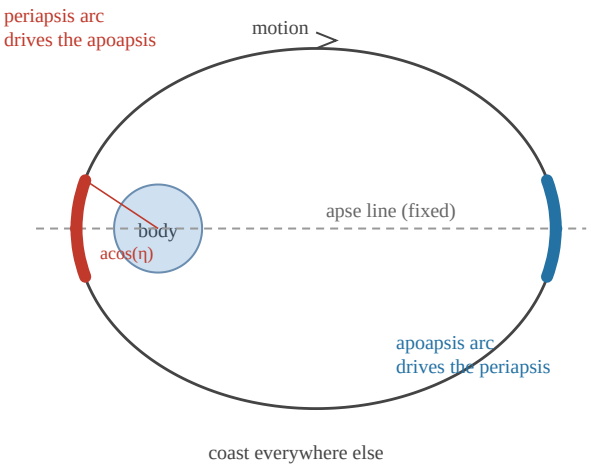
Apsides changes for a low-thrust vessel: the solver, the planner and the stand-in simulator. No kRPC side yet.

## What it does

Given the vessel's current orbit, a target apoapsis and/or periapsis (an apsis left out is free) and optionally a time budget, it decides every tick whether to thrust, which way (prograde or retrograde only) and how hard, until each targeted apsis is within `TOLERANCE_M` (100 m). The decision uses only the current orbit; there is no stored schedule, so a lost tick or a weak pass is corrected on a later pass by the same law.

| module | role |
|---|---|
| `orbit.py` | elements from a state, exact coasting (f and g series), radius toward a fixed direction, time to a true anomaly, the two-burn impulsive reference |
| `guidance.py` | the law: `lock_apse`, `command`, `arrived` |
| `plan.py` | flies the law in the simulator to pick the arc width that meets the deadline |
| `sim.py` | the stand-in: point mass, constant-thrust engine, spherical planet (Kerbin's numbers) |
| `tools/simfly.py` | command line: plan, fly, report dv against the impulsive Hohmann and the burn list |

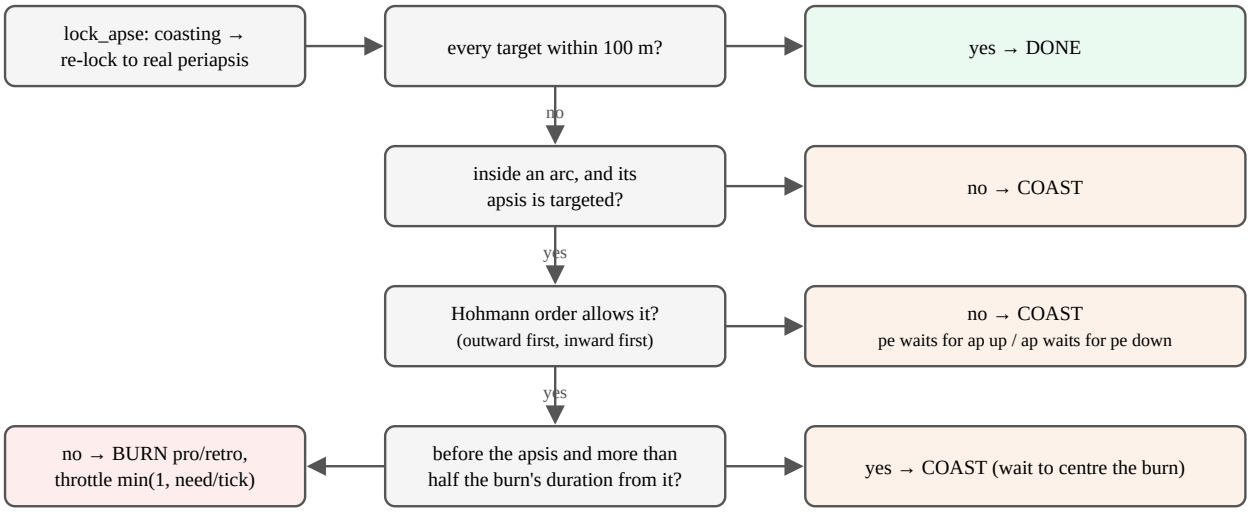
## The law: the Hohmann transfer in many small burns




Two arcs, centred on a fixed apse line. Each fixes the opposite apsis.

- **Decoupled.** A tangential burn at periapsis moves the apoapsis and, to first order, not the periapsis; one at apoapsis moves only the periapsis. So the periapsis arc drives the apoapsis to its target and the apoapsis arc drives the periapsis. The last pass nulls its element exactly.
- **Radius toward a fixed direction.** The quantity driven is the orbit's radius toward the far end of the apse line, `r = p / (1 + e·u)`, smooth through circular. Its rate per m/s of prograde is differenced numerically (`RATE_DV`).
- **Arc half-width** is `acos(η)`; η is a tangential burn's effectivity off-apsis. Default η = 0.99 (±8°).
- **Prograde or retrograde only**, so plain SAS modes can hold it.

## What triggers what, each tick




One tick of `guidance.command`. "need" is the burn time at full thrust to null the driven radius.

- **Apse line.** Held through each burn; re-locked to the osculating periapsis whenever the vessel is coasting. On a circular orbit with no line yet, the vessel's position sets it.
- **Burn timing.** A burn begins half its remaining duration before the apsis and runs at full throttle until its radius is on target or the arc ends; the final tick's throttle is the fraction still needed. No pass is longer than its arc.
- **Sign.** Prograde if raising the far radius closes the error, retrograde otherwise. The acceleration used is the one the engine gives now, never the rated figure.

## The planner and the time budget

Narrow arcs are many small, near-impulsive burns: cheapest and slowest. Wide arcs deliver more per pass and finish sooner for more dv. `plan.plan` flies the law in the simulator at `PLAN_GRID` (13) arc widths spaced evenly in half-width between `ETA_MAX` (0.999, ±2.6°) and `ETA_MIN` (0.2, ±78°), and takes the narrowest whose flight arrives within the budget. Without a budget it flies `ETA_DEFAULT` (0.99). If no width meets the budget it reports so and returns the fastest transfer. The prediction (dv, time, xenon, burn list) is the law itself flown forward, so it describes what the vessel will fly.

## The stand-in simulator

- Two-body gravity about Kerbin (μ 3.5316e12, radius 600 km, atmosphere 70 km); a vessel of dry mass, propellant, thrust and Isp (default 4 t + 1 t xenon, one 2 kN / Isp 4200 engine).
- A tick is period / `TICKS_PER_ORBIT` (180). Guidance is called once a tick and its command held. Coasting ticks are propagated exactly; thrusting ticks by `THRUST_SUBSTEPS` (4) RK4 steps, mass falling at thrust / (Isp g0).
- A flight ends on arrival, escape, periapsis inside the atmosphere, empty tanks, or `MAX_ORBITS`.